



Universidade Estadual de Santa Cruz – UESC

**Relatórios de Implementações de Métodos da Disciplina
Análise Numérica**

**Relatório de implementações
realizadas por Ronaldo Ribeiro Porto
Filho**

Disciplina Análise Numérica.

Curso Ciência da Computação

Semestre 2026.1

Professor Gesil Sampaio Amarante II

**Ilhéus – BA
2026**

ÍNDICE

ÍNDICE.....	2
Linguagem(ns) Escolhida(s) e justificativas.....	5
Regressão Linear.....	6
Estratégia de Implementação:.....	6
Estrutura dos Arquivos de Entrada/Saída.....	6
Dificuldades enfrentadas.....	6
Aproximação Polinomial Discreta.....	8
Estratégia de Implementação:.....	8
Estrutura dos Arquivos de Entrada/Saída.....	8
Dificuldades enfrentadas.....	8
Aproximação Polinomial Contínua.....	9
Estratégia de Implementação:.....	9
Estrutura dos Arquivos de Entrada/Saída.....	9
Dificuldades enfrentadas.....	9
Interpolação Polinomial de Lagrange.....	10
Estratégia de Implementação:.....	10
Estrutura dos Arquivos de Entrada/Saída.....	10
Dificuldades enfrentadas.....	10
Interpolação Polinomial de Newton.....	11
Estratégia de Implementação:.....	11
Estrutura dos Arquivos de Entrada/Saída.....	11
Dificuldades enfrentadas.....	11
Problema Teste 1 - Página 268 Questão 8.1.....	12
Entrada 1 – para Regressão Linear.....	12
Entrada 2 – para Aproximação Polinomial Discreta.....	12
Saída – Regressão Linear (Entrada 1).....	13
Saída – Aproximação Polinomial Discreta (Entrada 2).....	13
Problema Teste 2 - Página 269 Questão 8.5.....	15
Entrada 1 – para Aproximação Polinomial Discreta.....	15
Saída – Aproximação Polinomial Discreta (Entrada 1).....	16
Problema Teste 3 - Página 273 Questão 8.11.....	17
Entrada 1 – para Regressão Linear.....	17
Entrada 2 – para Regressão Linear.....	17
Saída – Regressão Linear (Entrada 1).....	18
Saída – Regressão Linear (Entrada 2).....	18
Problema Teste 4 - Página 316 Questão 10.2.....	19
Entrada 1 – para Interpolação Polinomial de Lagrange.....	19
Entrada 1 – para Interpolação Polinomial de Newton.....	19
Saída – Interpolação Polinomial de Lagrange (Entrada 1).....	20

Saída – Interpolação Polinomial de Newton (Entrada 1).....	21
Problema Teste 5 - Página 318 Questão 10.6.....	22
Entrada 1 – para Interpolação Polinomial de Lagrange.....	22
Entrada 1 – para Interpolação Polinomial de Newton.....	22
Entrada 2 – para Interpolação Polinomial de Lagrange.....	22
Entrada 2 – para Interpolação Polinomial de Newton.....	22
Entrada 3 – para Interpolação Polinomial de Lagrange.....	22
Entrada 3 – para Interpolação Polinomial de Newton.....	23
Entrada 4 – para Interpolação Polinomial de Lagrange.....	23
Entrada 4 – para Interpolação Polinomial de Newton.....	23
Saída – Interpolação Polinomial de Lagrange (Entrada 1).....	24
Saída – Interpolação Polinomial de Newton (Entrada 1).....	25
Saída – Interpolação Polinomial de Lagrange (Entrada 2).....	26
Saída – Interpolação Polinomial de Newton (Entrada 2).....	27
Saída – Interpolação Polinomial de Lagrange (Entrada 3).....	28
Saída – Interpolação Polinomial de Newton (Entrada 3).....	29
Saída – Interpolação Polinomial de Lagrange (Entrada 4).....	30
Saída – Interpolação Polinomial de Newton (Entrada 4).....	31
Problema Teste 6 - Página 318 Questão 10.9.....	33
Entrada 1 – para Interpolação Polinomial de Lagrange.....	34
Entrada 1 – para Interpolação Polinomial de Newton.....	34
Entrada 2 – para Interpolação Polinomial de Lagrange.....	34
Entrada 2 – para Interpolação Polinomial de Newton.....	34
Saída – Interpolação Polinomial de Lagrange (Entrada 1).....	35
Saída – Interpolação Polinomial de Newton (Entrada 1).....	40
Saída – Interpolação Polinomial de Lagrange (Entrada 2).....	41
Saída – Interpolação Polinomial de Newton (Entrada 2).....	42
Derivação Numérica de Primeira Ordem.....	44
Estratégia de Implementação:.....	44
Estrutura dos Arquivos de Entrada/Saída.....	44
Dificuldades enfrentadas.....	44
Derivação Numérica de Segunda Ordem.....	45
Estratégia de Implementação:.....	45
Estrutura dos Arquivos de Entrada/Saída.....	45
Dificuldades enfrentadas.....	45
Integração por Trapézios Simples e Compostos.....	46
Estratégia de Implementação:.....	46
Estrutura dos Arquivos de Entrada/Saída.....	46
Dificuldades enfrentadas.....	46
Integração por Simpson 1/3.....	47
Estratégia de Implementação:.....	47
Estrutura dos Arquivos de Entrada/Saída.....	47

Dificuldades enfrentadas.....	47
Integração por Simpson 3/8.....	48
Estratégia de Implementação:.....	48
Estrutura dos Arquivos de Entrada/Saída.....	48
Dificuldades enfrentadas.....	48
Integração com Extrapolação de Richardson.....	49
Estratégia de Implementação:.....	49
Estrutura dos Arquivos de Entrada/Saída.....	49
Dificuldades enfrentadas.....	49
Integração com Quadratura de Gauss.....	50
Estratégia de Implementação:.....	50
Estrutura dos Arquivos de Entrada/Saída.....	50
Dificuldades enfrentadas.....	50
Problema Teste 7 - Página 366 Questão 11.1.....	51
Entrada 1 – para Integração com Simpson 1/3.....	51
Saída – Integração com Simpson 1/3 (Entrada 1).....	52
Problema Teste 8 - Página 368 Questão 11.6.....	53
Entrada 1 – para Integração com Simpson 1/3.....	53
Saída – Integração com Simpson 1/3 (Entrada 1).....	54
Problema Teste 9 - Página 371 Questão 11.11.....	55
Entrada 1 – para Integração com Extrapolação de Richardson.....	56
Saída – Integração com Extrapolação de Richardson (Entrada 1).....	56
Considerações Finais.....	58

Linguagem(ns) Escolhida(s) e justificativas

A linguagem escolhida para o desenvolvimento de todos os métodos numéricos deste relatório foi o Python. A decisão principal baseia-se em sua sintaxe simples e de alto nível, que se assemelha muito à notação matemática convencional. Isso facilitou muito a tradução direta das fórmulas teóricas para os algoritmos, minimizando erros lógicos. Além disso, trata-se da linguagem recomendada pelo professor para a disciplina.

Para a implementação, a estratégia foi priorizar a biblioteca padrão do Python para manter o código leve. Foi utilizada a biblioteca integrada math para as operações algébricas e trigonométricas, e a copy para a manipulação segura de matrizes durante a Eliminação de Gauss. A biblioteca externa NumPy foi adotada pontualmente apenas no módulo de Interpolação, devido à sua classe `poly1d`, que simplifica a complexa multiplicação de binômios nos métodos de Lagrange e Newton.

Outro ponto fundamental foi a facilidade do Python para a leitura e escrita de arquivos de texto simples (`entrada.txt` e `saida.txt`). Aliado ao uso do comando `eval()` em um ambiente de execução restrito e seguro, foi possível ler funções dinamicamente e facilitar a geração de relatórios com o passo a passo de cada cálculo.

A principal limitação prática foi lidar com a aritmética de ponto flutuante da linguagem; operações sucessivas (como no sistema linear do Método dos Mínimos Quadrados) geravam pequenos ruídos de arredondamento (resíduos na ordem de 10^{-15}). Para contornar isso e manter os relatórios legíveis, foi necessário implementar lógicas de formatação de strings que filtrassem essas imprecisões numéricas antes da gravação nos arquivos de saída.

Regressão Linear

Estratégia de Implementação:

A implementação da Regressão Linear foi estruturada para calcular a reta $f(x) = a_1x + a_0$ que melhor se ajusta aos dados numéricos pelo Método dos Mínimos Quadrados. O algoritmo foi desenvolvido em Python puro, onde percorre a lista de coordenadas em um único laço de repetição para acumular os somatórios necessários de forma direta e eficiente, dispensando o uso de bibliotecas externas pesadas.

Para garantir a estabilidade do algoritmo, foram inseridas validações estruturais. O código verifica se há pelo menos dois pontos disponíveis e calcula previamente o denominador da fórmula; caso seja zero (indicando pontos colineares verticalmente), o sistema aborta a operação de forma segura com um log de erro. Também inclui um tratamento na formatação da string final para garantir que o sinal da equação saia matematicamente correto.

Estrutura dos Arquivos de Entrada/Saída

Arquivo de Entrada (entrada.txt): estruturado de forma minimalista, contém um par ordenado por linha, com os valores de x e y separados apenas por um espaço (exemplo: 1.5 3.2). Essa abordagem simples permite a leitura limpa dos dados pela função nativa do Python, evitando falhas comuns de formatação com vírgulas.

Arquivo de Saída (saida.txt): atua como uma memória de cálculo passo a passo. O arquivo não entrega apenas a equação final da reta, mas registra o formato inicial dos dados processados (agrupados por parênteses) e os valores exatos de cada somatório intermediário, facilitando muito a validação teórica e a conferência manual dos resultados.

Dificuldades enfrentadas

A principal dificuldade matemática na implementação da regressão linear foi o tratamento da instabilidade gerada por divisões por zero. Em cenários onde os dados fornecidos são colineares verticalmente (ou seja,

possuem o mesmo valor para a coordenada x), o denominador da fórmula do Método dos Mínimos Quadrados se anula, o que causaria o travamento abrupto do programa. Para resolver isso, foi necessário implementar uma validação prévia que interrompe o cálculo de forma segura nessas condições.

Outro desafio prático envolveu a formatação da equação final no arquivo de saída. Como o coeficiente linear pode assumir valores negativos, a impressão direta dos números frequentemente gerava anomalias visuais. Isso exigiu a criação de uma lógica condicional específica para manipular os sinais algébricos antes da gravação. Além disso, garantir que o sistema de leitura do arquivo “entrada.txt” ignorasse quebras de linha acidentais e isolasse os pares ordenados de forma limpa demandou atenção na manipulação de strings.

Aproximação Polinomial Discreta

Estratégia de Implementação:

A implementação da aproximação polinomial discreta (Método dos Mínimos Quadrados para graus maiores que 1) baseou-se na construção de um sistema de equações normais. O algoritmo percorre os dados calculando as somatórias de x elevado a potências progressivas para preencher a matriz de coeficientes, e os produtos de y por x para o vetor independente. A principal escolha arquitetural aqui foi reaproveitar a função de Eliminação de Gauss, construída isoladamente no código, para resolver o sistema linear resultante. Isso manteve o programa modular e eliminou a necessidade de usar bibliotecas pesadas de álgebra linear.

Estrutura dos Arquivos de Entrada/Saída

A estrutura do arquivo de entrada (`entrada.txt`) permanece idêntica à da regressão linear, contendo um par ordenado por linha com os valores separados por espaço simples. A manutenção desse formato unificado facilita a transição entre os métodos sem precisar alterar a base de dados original. O arquivo de saída (`saida.txt`) registra não apenas a resposta, mas imprime a matriz das equações normais gerada e o vetor independente antes da resolução, atuando como um memorial de cálculo.

Dificuldades enfrentadas

A maior dificuldade na implementação foi o tratamento da exibição do polinômio final. Como os coeficientes resolvidos pelo sistema de Gauss podem ser negativos ou muito próximos de zero, foi necessário desenvolver uma função auxiliar apenas para montar a string da equação, ajustando os sinais corretamente para não gerar anomalias visuais no arquivo de saída. Além disso, observou-se que tentar forçar polinômios de graus muito altos com essa abordagem gera somatórias enormes, o que pode causar instabilidade numérica na matriz devido às limitações das casas decimais.

Aproximação Polinomial Contínua

Estratégia de Implementação:

A implementação da aproximação polinomial contínua baseou-se no Método dos Mínimos Quadrados, substituindo os somatórios discretos por integrais definidas ao longo de um intervalo. A matriz dos coeficientes foi calculada diretamente pela integração analítica programada das potências de x . Já para o vetor independente, que exige a integração do produto da função avaliada por x , optei por não utilizar bibliotecas de cálculo simbólico pesadas (como o SymPy). Em vez disso, implementei internamente a Regra de Simpson 1/3 para resolver essa integral numericamente. O sistema linear resultante foi resolvido reaproveitando a função modular de Eliminação de Gauss.

Estrutura dos Arquivos de Entrada/Saída

O arquivo de entrada (entrada.txt) precisou ter sua estrutura modificada para este caso específico. Em vez de coordenadas, cada linha recebe a expressão matemática da função e seus limites de integração, separados por ponto e vírgula e vírgula (exemplo: $\cos(x)$; 0, 1.5). O arquivo de saída (saida.txt) mantém a função de memorial detalhado, imprimindo a função lida, a matriz de integrais calculada, o vetor numérico gerado por Simpson e, finalmente, o polinômio resultante devidamente formatado.

Dificuldades enfrentadas

A maior dificuldade nesta etapa foi lidar com a interpretação de funções matemáticas inseridas como texto puro. Para calcular as integrais numericamente, foi necessário usar o comando `eval` do Python em um ambiente restrito, garantindo que as expressões fossem avaliadas com segurança a cada iteração do método de Simpson. Além disso, a combinação de aproximação de grau elevado com a integração numérica gera matrizes com valores muito altos ou muito próximos de zero, o que pode amplificar erros de arredondamento durante a Eliminação de Gauss, exigindo cautela na escolha do grau do polinômio desejado.

Interpolação Polinomial de Lagrange

Estratégia de Implementação:

A implementação do método de Lagrange teve como objetivo encontrar um único polinômio que passe exatamente sobre todos os pontos do conjunto de dados. A estratégia central consistiu em calcular os polinômios base $L_i(x)$ iterativamente. Para lidar com a álgebra complexa de multiplicar binômios literais (como multiplicar $x - x_0$ por $x - x_1$), utilizei a classe `poly1d` da biblioteca externa NumPy. Essa ferramenta permitiu realizar a propriedade distributiva e o agrupamento de graus polinomiais de forma nativa e eficiente, somando os termos no final para compor a equação interpoladora.

Estrutura dos Arquivos de Entrada/Saída

O arquivo de entrada (`entrada.txt`) manteve a padronização dos métodos de aproximação, recebendo os pares de coordenadas x e y separados por um espaço simples em cada linha. O arquivo de saída (`saida.txt`) foi estruturado como um memorial de cálculo passo a passo: ele não entrega apenas a resposta, mas registra a construção matemática e a equação expandida de cada termo base $L_i(x)$ individualmente, facilitando o acompanhamento da lógica antes de exibir a soma total.

Dificuldades enfrentadas

A principal dificuldade superada foi justamente a expansão algébrica, que geraria um código massivo e ilegível se desenvolvida do zero em Python puro. Embora a adoção do NumPy tenha resolvido as multiplicações, ela trouxe um problema devido à aritmética de ponto flutuante da linguagem, que frequentemente exibía coeficientes residuais indesejados. A solução exigiu o desenvolvimento de uma função dedicada à formatação de texto (`formatar_poly1d`) para filtrar essas imprecisões e garantir uma saída limpa no relatório final.

Interpolação Polinomial de Newton

Estratégia de Implementação:

A implementação da Interpolação de Newton baseou-se no método das Diferenças Divididas. A estratégia central foi construir uma matriz em memória para calcular sucessivamente as diferenças de ordem superior, onde a primeira linha dessa matriz fornece os coeficientes da equação final. Para contornar a complexidade de multiplicar os termos binomiais progressivos, como multiplicar o coeficiente por $(x - x_0)$ e depois por $(x - x_1)$, recorreu-se à classe `poly1d` da biblioteca NumPy. Isso permitiu aplicar a propriedade distributiva de forma nativa e acumular o polinômio principal com alta eficiência algébrica.

Estrutura dos Arquivos de Entrada/Saída

A estrutura do arquivo de entrada (`entrada.txt`) seguiu o padrão estabelecido nos outros métodos, lendo pares de coordenadas x e y separados por espaço simples. O diferencial ocorreu no arquivo de saída (`saida.txt`). Em vez de apenas apresentar a equação pronta, o código foi desenhado para atuar como um memorial prático, formatando e imprimindo visualmente a Tabela de Diferenças Divididas completa antes de exibir a construção passo a passo do polinômio, facilitando a conferência teórica dos cálculos.

Dificuldades enfrentadas

A principal dificuldade nesta implementação foi o gerenciamento lógico dos laços de repetição aninhados para preencher a matriz de diferenças sem estourar os limites das listas (`index out of range`). Além disso, o uso do NumPy para as multiplicações algébricas gerou o mesmo desafio do método anterior: o aparecimento de imprecisões residuais de ponto flutuante. A solução foi aplicar a mesma função de formatação de texto desenvolvida para Lagrange, que filtra esses ruídos próximos de zero e organiza a string final para que o polinômio seja exibido de forma limpa e correta matematicamente.

Problema Teste 1 - Página 268 Questão 8.1

8.1 - A tabela a seguir lista o número de acidentes em veículos motorizados no Brasil em alguns anos entre 1980 e 1998.

ano	nº de acidentes (em milhares)	acidentes por 10000 veículos
1980	8.300	1.688
1985	9.900	1.577
1990	10.400	1.397
1993	13.200	1.439
1994	13.600	1.418
1996	13.700	1.385
1998	14.600	1.415

a) Calcule a regressão linear do número de acidentes no tempo. Use-a para prever o número de acidentes no ano 2000. (Isto é chamada uma análise de série temporal, visto que é uma regressão no tempo e é usada para prognosticar o futuro).

b) Calcule uma regressão quadrática do número de acidentes por 10000 veículos. Use esta para prognosticar o número de acidentes por 10000 veículos no ano 2000.

c) Compare os resultados da parte a) e b). Em qual delas você está mais propenso a acreditar?

Observe que em qualquer trabalho de série temporal envolvendo datas contemporâneas é uma boa idéia eliminar os dados iniciais antes de formar as somas; isto reduzirá os problemas de arredondamento. Assim, em vez de usar para x os valores 1980, 1985, etc, usamos 0, 5, etc.

Entrada 1 – para Regressão Linear

1	0	8.300
2	5	9.900
3	10	10.400
4	13	13.200
5	14	13.600
6	16	13.700
7	18	14.600

Entrada 2 – para Aproximação Polinomial Discreta

1	0	1.688
2	5	1.577
3	10	1.397
4	13	1.439
5	14	1.418
6	16	1.385
7	18	1.415

Saída – Regressão Linear (Entrada 1)

```
1  Lista de pontos: ( (0.0, 8.3), (5.0, 9.9), (10.0, 10.4), (13.0, 13.2), (14.0, 13.6), (16.0, 13.7), (18.0, 14.6) )
2
3  ===== RELATORIO: REGRESSAO LINEAR =====
4
5  --- FASE 1: CALCULO DOS SOMATORIOS ---
6
7  =====
8  RESULTADO FINAL - EQUACAO ENCONTRADA:
9  f(x) = 8.0216 + 0.3625x
10 =====
```

Saída – Aproximação Polinomial Discreta (Entrada 2)

```
1  Lista de pontos: ( (0.0, 1.688), (5.0, 1.577), (10.0, 1.397),
2  (13.0, 1.439), (14.0, 1.418), (16.0, 1.385), (18.0, 1.415) )
3
4  ===== RELATORIO: APROXIMACAO DISCRETA (Grau 2) =====
5
6  --- FASE 1: EQUACOES NORMAIS DISCRETAS (Grau 2) ---
7
8  Matriz dos Coeficientes (Somatorios de X):
9  [7.0, 76.0, 1070.0]
10 [76.0, 1070.0, 15994.0]
11 [1070.0, 15994.0, 248114.0]
12
13 Vetor Independente (Somatorios de Y * X^i):
14 [10.319, 108.044, 1513.264]
15
16 [SISTEMA LINEAR] --- FASE 1: ELIMINACAO PROGRESSIVA ---
17 Linha 2 = Linha 2 - (10.8571) * Linha 1
18 Linha 3 = Linha 3 - (152.8571) * Linha 1
19 Linha 3 = Linha 3 - (17.8751) * Linha 2
20
21 [SISTEMA LINEAR] --- FASE 2: SUBSTITUICAO REGRESSIVA ---
22
23 =====
24 RESULTADO FINAL - EQUACAO ENCONTRADA:
25 f(x) = 1.6985 -0.0369x + 0.0012x^2
26 =====
```

Resposta 8.1:

a) Para o ano 2000 basta substituir x por 20:

$$8,0216 + 0,3625x =$$

$$8,0216 + 0,3625 \cdot 20 =$$

$$\mathbf{15,2716}$$

b) Para o ano 2000 basta substituir x por 20:

$$1,6985 - 0,0369x + 0,0012x^2 =$$

$$1,6985 - 0,0369 \cdot 20 + 0,0012 \cdot 20^2 =$$

$$1,6985 - 0,0369 \cdot 20 + 0,0012 \cdot 20^2 =$$

$$1,6985 - 0,738 + 0,48 =$$

$$1,6985 - 0,258 =$$

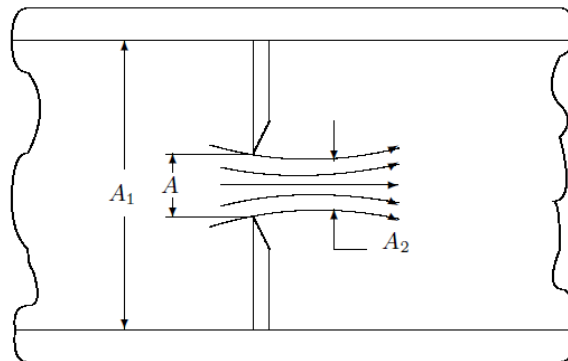
$$\mathbf{1,4405}$$

c) Estou mais propenso a acreditar na letra b, porque a letra a tem um salto um pouco maior do que o normal quando comparado com valores anteriores, e isso não ocorre com a letra b.

Problema Teste 2 - Página 269 Questão 8.5

8.5 -Placas de orifício com bordas em canto (ou faca) são muito utilizadas na medição da vazão de fluidos através de tubulações . A figura a seguir mostra uma placa de orifício, que tem os seguintes parâmetros geométricos representativos:

- A = área da seção reta do orifício
- A_1 = área da seção reta da tubulação
- $A_2 = CA$ (seção reta no ponto de maior contração após o orifício)



O coeficiente C é função da razão A/A_1 , e valores experimentais desse coeficiente estão listados na tabela a seguir:

A/A_1	0.10	0.20	0.30	0.40	0.50	0.60	0.70	0.80	0.90	1.00
C	0.62	0.63	0.64	0.66	0.68	0.71	0.76	0.81	0.89	1.00

a) Fazendo $x = A/A_1$, ajuste a função $C(x)$ pela função: $a_0 + a_1x + a_2x^2$ aos pontos da tabela, usando o método dos mínimos quadrados.

Entrada 1 – para Aproximação Polinomial Discreta

1	0.10	0.62
2	0.20	0.63
3	0.30	0.64
4	0.40	0.66
5	0.50	0.68
6	0.60	0.71
7	0.70	0.76
8	0.80	0.81
9	0.90	0.89
10	1.00	1.00

Saída – Aproximação Polinomial Discreta (Entrada 1)

```
1  Lista de pontos: ( (0.1, 0.62), (0.2, 0.63), (0.3, 0.64),
2  (0.4, 0.66), (0.5, 0.68), (0.6, 0.71), (0.7, 0.76),
3  (0.8, 0.81), (0.9, 0.89), (1.0, 1.0) )
4
5  ===== RELATORIO: APROXIMACAO DISCRETA (Grau 2) =====
6
7  --- FASE 1: EQUACOES NORMAIS DISCRETAS (Grau 2) ---
8
9  Matriz dos Coeficientes (Somatorios de X):
10 [10.0, 5.5, 3.85]
11 [5.5, 3.85, 3.025]
12 [3.85, 3.025, 2.5333]
13
14 Vetor Independente (Somatorios de Y * X^i):
15 [7.4, 4.391, 3.2319]
16
17 [SISTEMA LINEAR] --- FASE 1: ELIMINACAO PROGRESSIVA ---
18 Linha 2 = Linha 2 - (0.5500) * Linha 1
19 Linha 3 = Linha 3 - (0.3850) * Linha 1
20 Linha 3 = Linha 3 - (1.1000) * Linha 2
21
22 [SISTEMA LINEAR] --- FASE 2: SUBSTITUICAO REGRESSIVA ---
23
24 =====
25 RESULTADO FINAL - EQUACAO ENCONTRADA:
26 f(x) = 0.6502 - 0.2317x + 0.5644x^2
27 =====
```

Resposta 8.5:

a) $0,6502 - 0,2317x + 0,5644x^2$

Problema Teste 3 - Página 273 Questão 8.11

8.11 - A tabela a seguir lista o Produto Nacional Bruto (PNB) em dólares constantes e correntes. Os dólares constantes representam o PNB baseado no valor do dólar em 1987. Os dólares correntes são simplesmente o valor sem nenhum ajuste de inflação.

ano	PNB (dólar corrente) (em milhões)	PNB (dólar constante) (em milhões)
1980	248.8	355.3
1985	398.0	438.0
1989	503.7	487.7
1992	684.9	617.8
1994	749.9	658.1
1995	793.5	674.6
1997	865.7	707.6

Estudos mostram que a melhor forma de trabalhar os dados é aproximá-los por uma função do tipo: $a x^b$. Assim:

- Utilize a função $a x^b$ para cada um dos PNB's no tempo.
- Use os resultados da parte a) para prever os PNB's no ano 2000.

Entrada 1 - para Regressão Linear

1	0.0000	5.5166
2	1.7917	5.9864
3	2.3025	6.2219
4	2.5649	6.5292
5	2.7080	6.6200
6	2.7725	6.6764
7	2.8903	6.7635

Entrada 2 - para Regressão Linear

1	0.0000	5.8729
2	1.7917	6.0822
3	2.3025	6.1897
4	2.5649	6.4261
5	2.7080	6.4893
6	2.7725	6.5141
7	2.8903	6.5618

Saída – Regressão Linear (Entrada 1)

```
1  Lista de pontos: ( (0.0, 5.5166), (1.7917, 5.9864),
2  (2.3025, 6.2219), (2.5649, 6.5292), (2.708, 6.62),
3  (2.7725, 6.6764), (2.8903, 6.7635) )
4
5  ===== RELATORIO: REGRESSAO LINEAR =====
6
7  --- FASE 1: CALCULO DOS SOMATORIOS ---
8
9  =====
10 RESULTADO FINAL - EQUACAO ENCONTRADA:
11 f(x) = 5.4165 + 0.4257x
12 =====
```

Saída – Regressão Linear (Entrada 2)

```
1  Lista de pontos: ( (0.0, 5.8729), (1.7917, 6.0822),
2  (2.3025, 6.1897), (2.5649, 6.4261), (2.708, 6.4893),
3  (2.7725, 6.5141), (2.8903, 6.5618) )
4
5  ===== RELATORIO: REGRESSAO LINEAR =====
6
7  --- FASE 1: CALCULO DOS SOMATORIOS ---
8
9  =====
10 RESULTADO FINAL - EQUACAO ENCONTRADA:
11 f(x) = 5.7974 + 0.2365x
12 =====
```

Resposta 8.11:

a)

Dólar Corrente: $y = e^{5,4165}x^{0,4257}$

Dólar Constante: $y = e^{5,7974}x^{0,2365}$

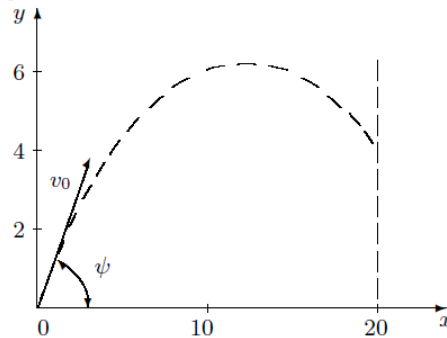
b)

Dólar Corrente para o ano 2000 ($x = 21$): $y = e^{5,4165}21^{0,4257} = \mathbf{822.66}$

Dólar Constante para o ano 2000 ($x = 21$): $y = e^{5,7974}21^{0,2365} = \mathbf{676.83}$

Problema Teste 4 - Página 316 Questão 10.2

10.2 - Um projétil foi lançado de um ponto tomado como origem e fez-se as seguintes observações:



isto é:

- i) fotografou-se o projétil a 10 metros do ponto de lançamento e foi determinado sua altitude no local: 6 metros.
- ii) uma barreira a 20 metros do ponto de lançamento interceptou-o e aí foi determinada sua altitude: 4 metros.

Com esses 3 pontos é possível interpolar a trajetória do projétil. Comparando a equação teórica da trajetória com a obtida pela interpolação é possível determinar os parâmetros de lançamento: o ângulo ψ com a horizontal, e a velocidade inicial v_0 . Assim:

- a) Determine o polinômio interpolador.
- b) Determine ψ e v_0 sabendo que a equação da trajetória é dada por:

$$y = x \operatorname{tg} \psi - \frac{1}{2} g \frac{x^2}{v_0^2 \cos^2 \psi},$$

onde $g = 9.86 \text{ m/s}^2$.

- c) Calcule a altitude do projétil a 5 metros do ponto de lançamento.

Entrada 1 – para Interpolação Polinomial de Lagrange

1	0	0
2	10	6
3	20	4

Entrada 1 – para Interpolação Polinomial de Newton

1	0	0
2	10	6
3	20	4

Saída – Interpolação Polinomial de Lagrange (Entrada 1)

```
1  Conjunto de dados lido: ((0.0, 0.0), (10.0, 6.0), (20.0, 4.0))
2
3  ===== RELATORIO: INTERPOLACAO DE LAGRANGE =====
4
5  --- FASE 1: CONSTRUCAO DOS POLINOMIOS BASE L_i(x) ---
6
7  ∨ Calculando L_0(x) para o ponto (0.0, 0.0):
8      Multiplicando por (x - 10.0) / (0.0 - 10.0)
9      Multiplicando por (x - 20.0) / (0.0 - 20.0)
10     L_0(x) resultante = 0.0050x^2 - 0.1500x + 1.0000
11     Adicionando ao polinomio principal: 0.0 * L_0(x)
12
13  ∨ Calculando L_1(x) para o ponto (10.0, 6.0):
14     Multiplicando por (x - 0.0) / (10.0 - 0.0)
15     Multiplicando por (x - 20.0) / (10.0 - 20.0)
16     L_1(x) resultante = - 0.0100x^2 + 0.2000x
17     Adicionando ao polinomio principal: 6.0 * L_1(x)
18
19  ∨ Calculando L_2(x) para o ponto (20.0, 4.0):
20     Multiplicando por (x - 0.0) / (20.0 - 0.0)
21     Multiplicando por (x - 10.0) / (20.0 - 10.0)
22     L_2(x) resultante = 0.0050x^2 - 0.0500x
23     Adicionando ao polinomio principal: 4.0 * L_2(x)
24
25  =====
26  RESULTADO FINAL - POLINOMIO INTERPOLADOR UNICO:
27  P(x) = - 0.0400x^2 + 1.0000x
28  =====
```

Saída – Interpolação Polinomial de Newton (Entrada 1)

```
1  Conjunto de dados lido: ((0.0, 0.0), (10.0, 6.0), (20.0, 4.0))
2
3  ===== RELATORIO: INTERPOLACAO DE NEWTON =====
4
5  --- FASE 1: TABELA DE DIFERENCAS DIVIDIDAS ---
6  x          | Ordem 0  | Ordem 1  | ...
7  0.0000     | 0.0000   | 0.6000   | -0.0400
8  10.0000    | 6.0000   | -0.2000
9  20.0000    | 4.0000
10
11 --- FASE 2: CONSTRUCAO DO POLINOMIO ---
12 Coeficientes de Newton (primeira linha da tabela):
13 [0.0, 0.6, -0.04]
14
15 Passo 1: Adicionando 0.0000 * (1.0000)
16 Passo 2: Adicionando 0.6000 * (1.0000x)
17 Passo 3: Adicionando -0.0400 * (1.0000x^2 - 10.0000x)
18
19 =====
20 RESULTADO FINAL - POLINOMIO INTERPOLADOR UNICO:
21 P(x) = - 0.0400x^2 + 1.0000x
22 =====
```

Resposta 10.2:

a) $y(x) = -0,0400x^2 + 1,0000x$.

b) O ângulo é 45° e $v_0 = 15,70$ m/s.

c) $y(5) = 4$ metros.

Problema Teste 5 - Página 318 Questão 10.6

10.6 - Conhecendo-se o diâmetro e a resistividade de um fio cilíndrico verificou-se a resistência do fio de acordo com o comprimento. Os dados obtidos estão indicados a seguir:

Comp.(m)	500	1000	1500	2000	2500	3000	3500	4000
Res.(Ohms)	2.74	5.48	7.90	11.00	13.93	16.43	20.24	23.52

Determine quais serão as prováveis resistências deste fio para comprimentos de:

a) 1730 m,

b) 3200 m,

usando parábolas de 2º e 3º graus.

Entrada 1 – para Interpolação Polinomial de Lagrange

1	1000	5.48
2	1500	7.90
3	2000	11.00

Entrada 1 – para Interpolação Polinomial de Newton

1	1000	5.48
2	1500	7.90
3	2000	11.00

Entrada 2 – para Interpolação Polinomial de Lagrange

1	1000	5.48
2	1500	7.90
3	2000	11.00
4	2500	13.93

Entrada 2 – para Interpolação Polinomial de Newton

1	1000	5.48
2	1500	7.90
3	2000	11.00
4	2500	13.93

Entrada 3 – para Interpolação Polinomial de Lagrange

1	2500	13.93
2	3000	16.43
3	3500	20.24

Entrada 3 – para Interpolação Polinomial de Newton

1	2500	13.93
2	3000	16.43
3	3500	20.24

Entrada 4 – para Interpolação Polinomial de Lagrange

1	2500	13.93
2	3000	16.43
3	3500	20.24
4	4000	23.52

Entrada 4 – para Interpolação Polinomial de Newton

1	2500	13.93
2	3000	16.43
3	3500	20.24
4	4000	23.52

Saída – Interpolação Polinomial de Lagrange (Entrada 1)

```
1  Conjunto de dados lido: ((1000.0, 5.48), (1500.0, 7.9), (2000.0, 11.0))
2
3  ===== RELATORIO: INTERPOLACAO DE LAGRANGE =====
4
5  --- FASE 1: CONSTRUCAO DOS POLINOMIOS BASE L_i(x) ---
6
7  √ Calculando L_0(x) para o ponto (1000.0, 5.48):
8      Multiplicando por (x - 1500.0) / (1000.0 - 1500.0)
9      Multiplicando por (x - 2000.0) / (1000.0 - 2000.0)
10     L_0(x) resultante = 0.0000x^2 - 0.0070x + 6.0000
11     Adicionando ao polinomio principal: 5.48 * L_0(x)
12
13  √ Calculando L_1(x) para o ponto (1500.0, 7.9):
14     Multiplicando por (x - 1000.0) / (1500.0 - 1000.0)
15     Multiplicando por (x - 2000.0) / (1500.0 - 2000.0)
16     L_1(x) resultante = - 0.0000x^2 + 0.0120x - 8.0000
17     Adicionando ao polinomio principal: 7.9 * L_1(x)
18
19  √ Calculando L_2(x) para o ponto (2000.0, 11.0):
20     Multiplicando por (x - 1000.0) / (2000.0 - 1000.0)
21     Multiplicando por (x - 1500.0) / (2000.0 - 1500.0)
22     L_2(x) resultante = 0.0000x^2 - 0.0050x + 3.0000
23     Adicionando ao polinomio principal: 11.0 * L_2(x)
24
25  =====
26  RESULTADO FINAL - POLINOMIO INTERPOLADOR UNICO:
27  P(x) = 0.0000x^2 + 0.0014x + 2.6800
28  =====
```


Saída – Interpolação Polinomial de Newton (Entrada 1)

```
1  Conjunto de dados lido: ((1000.0, 5.48), (1500.0, 7.9), (2000.0, 11.0))
2
3  ===== RELATORIO: INTERPOLACAO DE NEWTON =====
4
5  --- FASE 1: TABELA DE DIFERENCAS DIVIDIDAS ---
6  x          | Ordem 0 | Ordem 1 | ...
7  1000.0000 | 5.4800 | 0.0048 | 0.0000
8  1500.0000 | 7.9000 | 0.0062
9  2000.0000 | 11.0000
10
11 --- FASE 2: CONSTRUCAO DO POLINOMIO ---
12 Coeficientes de Newton (primeira linha da tabela):
13 [5.48, 0.0048, 0.0]
14
15 Passo 1: Adicionando 5.4800 * (1.0000)
16 Passo 2: Adicionando 0.0048 * (1.0000x - 1000.0000)
17 Passo 3: Adicionando 0.0000 * (1.0000x^2 - 2500.0000x + 1500000.0000)
18
19 =====
20 RESULTADO FINAL - POLINOMIO INTERPOLADOR UNICO:
21  $P(x) = 0.0000x^2 + 0.0014x + 2.6800$ 
22 =====
```

Saída - Interpolação Polinomial de Lagrange (Entrada 2)

```
1  Conjunto de dados lido: ((1000.0, 5.48), (1500.0, 7.9),
2  (2000.0, 11.0), (2500.0, 13.93))
3
4  ===== RELATORIO: INTERPOLACAO DE LAGRANGE =====
5
6  --- FASE 1: CONSTRUCAO DOS POLINOMIOS BASE L_i(x) ---
7
8  Calculando L_0(x) para o ponto (1000.0, 5.48):
9      Multiplicando por (x - 1500.0) / (1000.0 - 1500.0)
10     Multiplicando por (x - 2000.0) / (1000.0 - 2000.0)
11     Multiplicando por (x - 2500.0) / (1000.0 - 2500.0)
12     L_0(x) resultante = - 0.0000x^3 + 0.0000x^2 - 0.0157x + 10.0000
13     Adicionando ao polinomio principal: 5.48 * L_0(x)
14
15  Calculando L_1(x) para o ponto (1500.0, 7.9):
16     Multiplicando por (x - 1000.0) / (1500.0 - 1000.0)
17     Multiplicando por (x - 2000.0) / (1500.0 - 2000.0)
18     Multiplicando por (x - 2500.0) / (1500.0 - 2500.0)
19     L_1(x) resultante = 0.0000x^3 - 0.0000x^2 + 0.0380x - 20.0000
20     Adicionando ao polinomio principal: 7.9 * L_1(x)
21
22  Calculando L_2(x) para o ponto (2000.0, 11.0):
23     Multiplicando por (x - 1000.0) / (2000.0 - 1000.0)
24     Multiplicando por (x - 1500.0) / (2000.0 - 1500.0)
25     Multiplicando por (x - 2500.0) / (2000.0 - 2500.0)
26     L_2(x) resultante = - 0.0000x^3 + 0.0000x^2 - 0.0310x + 15.0000
27     Adicionando ao polinomio principal: 11.0 * L_2(x)
28
29  Calculando L_3(x) para o ponto (2500.0, 13.93):
30     Multiplicando por (x - 1000.0) / (2500.0 - 1000.0)
31     Multiplicando por (x - 1500.0) / (2500.0 - 1500.0)
32     Multiplicando por (x - 2000.0) / (2500.0 - 2000.0)
33     L_3(x) resultante = 0.0000x^3 - 0.0000x^2 + 0.0087x - 4.0000
34     Adicionando ao polinomio principal: 13.93 * L_3(x)
35
36  =====
37  RESULTADO FINAL - POLINOMIO INTERPOLADOR UNICO:
38  P(x) = - 0.0000x^3 + 0.0000x^2 - 0.0059x + 6.0800
39  =====
```

Saída – Interpolação Polinomial de Newton (Entrada 2)

```
1  Conjunto de dados lido: ((1000.0, 5.48), (1500.0, 7.9), (2000.0, 11.0), (2500.0, 13.93))
2
3  ===== RELATORIO: INTERPOLACAO DE NEWTON =====
4
5  --- FASE 1: TABELA DE DIFERENCAS DIVIDIDAS ---
6  x          | Ordem 0 | Ordem 1 | ...
7  1000.0000 | 5.4800 | 0.0048 | 0.0000 | -0.0000
8  1500.0000 | 7.9000 | 0.0062 | -0.0000
9  2000.0000 | 11.0000 | 0.0059
10 2500.0000 | 13.9300
11
12 --- FASE 2: CONSTRUCAO DO POLINOMIO ---
13 Coeficientes de Newton (primeira linha da tabela):
14 [5.48, 0.0048, 0.0, -0.0]
15
16 Passo 1: Adicionando 5.4800 * (1.0000)
17 Passo 2: Adicionando 0.0048 * (1.0000x - 1000.0000)
18 Passo 3: Adicionando 0.0000 * (1.0000x^2 - 2500.0000x + 1500000.0000)
19 Passo 4: Adicionando -0.0000 * (1.0000x^3 - 4500.0000x^2 + 6500000.0000x - 3000000000.0000)
20
21 =====
22 RESULTADO FINAL - POLINOMIO INTERPOLADOR UNICO:
23 P(x) = - 0.0000x^3 + 0.0000x^2 - 0.0059x + 6.0800
24 =====
```

Saída – Interpolação Polinomial de Lagrange (Entrada 3)

```
1  Conjunto de dados lido: ((2500.0, 13.93), (3000.0, 16.43), (3500.0, 20.24))
2
3  ===== RELATORIO: INTERPOLACAO DE LAGRANGE =====
4
5  --- FASE 1: CONSTRUCAO DOS POLINOMIOS BASE L_i(x) ---
6
7  √ Calculando L_0(x) para o ponto (2500.0, 13.93):
8      Multiplicando por (x - 3000.0) / (2500.0 - 3000.0)
9      Multiplicando por (x - 3500.0) / (2500.0 - 3500.0)
10     L_0(x) resultante = 0.0000x^2 - 0.0130x + 21.0000
11     Adicionando ao polinomio principal: 13.93 * L_0(x)
12
13  √ Calculando L_1(x) para o ponto (3000.0, 16.43):
14     Multiplicando por (x - 2500.0) / (3000.0 - 2500.0)
15     Multiplicando por (x - 3500.0) / (3000.0 - 3500.0)
16     L_1(x) resultante = - 0.0000x^2 + 0.0240x - 35.0000
17     Adicionando ao polinomio principal: 16.43 * L_1(x)
18
19  √ Calculando L_2(x) para o ponto (3500.0, 20.24):
20     Multiplicando por (x - 2500.0) / (3500.0 - 2500.0)
21     Multiplicando por (x - 3000.0) / (3500.0 - 3000.0)
22     L_2(x) resultante = 0.0000x^2 - 0.0110x + 15.0000
23     Adicionando ao polinomio principal: 20.24 * L_2(x)
24
25  =====
26  RESULTADO FINAL - POLINOMIO INTERPOLADOR UNICO:
27  P(x) = 0.0000x^2 - 0.0094x + 21.0800
28  =====
```

Saída – Interpolação Polinomial de Newton (Entrada 3)

```
1  Conjunto de dados lido: ((2500.0, 13.93), (3000.0, 16.43), (3500.0, 20.24))
2
3  ===== RELATORIO: INTERPOLACAO DE NEWTON =====
4
5  --- FASE 1: TABELA DE DIFERENCAS DIVIDIDAS ---
6  x          | Ordem 0 | Ordem 1 | ...
7  2500.0000 | 13.9300 | 0.0050 | 0.0000
8  3000.0000 | 16.4300 | 0.0076
9  3500.0000 | 20.2400
10
11 --- FASE 2: CONSTRUCAO DO POLINOMIO ---
12 Coeficientes de Newton (primeira linha da tabela):
13 [13.93, 0.005, 0.0]
14
15 Passo 1: Adicionando 13.9300 * (1.0000)
16 Passo 2: Adicionando 0.0050 * (1.0000x - 2500.0000)
17 Passo 3: Adicionando 0.0000 * (1.0000x^2 - 5500.0000x + 7500000.0000)
18
19 =====
20 RESULTADO FINAL - POLINOMIO INTERPOLADOR UNICO:
21 P(x) = 0.0000x^2 - 0.0094x + 21.0800
22 =====
```

Saída – Interpolação Polinomial de Lagrange (Entrada 4)

```
1  Conjunto de dados lido: ((2500.0, 13.93), (3000.0, 16.43),
2  (3500.0, 20.24), (4000.0, 23.52))
3
4  ===== RELATORIO: INTERPOLACAO DE LAGRANGE =====
5
6  --- FASE 1: CONSTRUCAO DOS POLINOMIOS BASE L_i(x) ---
7
8  Calculando L_0(x) para o ponto (2500.0, 13.93):
9      Multiplicando por (x - 3000.0) / (2500.0 - 3000.0)
10     Multiplicando por (x - 3500.0) / (2500.0 - 3500.0)
11     Multiplicando por (x - 4000.0) / (2500.0 - 4000.0)
12     L_0(x) resultante = - 0.0000x^3 + 0.0000x^2 - 0.0487x + 56.0000
13     Adicionando ao polinomio principal: 13.93 * L_0(x)
14
15  Calculando L_1(x) para o ponto (3000.0, 16.43):
16     Multiplicando por (x - 2500.0) / (3000.0 - 2500.0)
17     Multiplicando por (x - 3500.0) / (3000.0 - 3500.0)
18     Multiplicando por (x - 4000.0) / (3000.0 - 4000.0)
19     L_1(x) resultante = 0.0000x^3 - 0.0000x^2 + 0.1310x - 140.0000
20     Adicionando ao polinomio principal: 16.43 * L_1(x)
21
22  Calculando L_2(x) para o ponto (3500.0, 20.24):
23     Multiplicando por (x - 2500.0) / (3500.0 - 2500.0)
24     Multiplicando por (x - 3000.0) / (3500.0 - 3000.0)
25     Multiplicando por (x - 4000.0) / (3500.0 - 4000.0)
26     L_2(x) resultante = - 0.0000x^3 + 0.0000x^2 - 0.1180x + 120.0000
27     Adicionando ao polinomio principal: 20.24 * L_2(x)
28
29  Calculando L_3(x) para o ponto (4000.0, 23.52):
30     Multiplicando por (x - 2500.0) / (4000.0 - 2500.0)
31     Multiplicando por (x - 3000.0) / (4000.0 - 3000.0)
32     Multiplicando por (x - 3500.0) / (4000.0 - 3500.0)
33     L_3(x) resultante = 0.0000x^3 - 0.0000x^2 + 0.0357x - 35.0000
34     Adicionando ao polinomio principal: 23.52 * L_3(x)
35
36  =====
37  RESULTADO FINAL - POLINOMIO INTERPOLADOR UNICO:
38  P(x) = - 0.0000x^3 + 0.0000x^2 - 0.0750x + 85.4800
39  =====
```

Saída – Interpolação Polinomial de Newton (Entrada 4)

```

1  Conjunto de dados lido: ((2500.0, 13.93), (3000.0, 16.43), (3500.0, 20.24), (4000.0, 23.52))
2
3  ===== RELATORIO: INTERPOLACAO DE NEWTON =====
4
5  --- FASE 1: TABELA DE DIFERENCAS DIVIDIDAS ---
6  x      | Ordem 0 | Ordem 1 | ...
7  2500.0000 | 13.9300 | 0.0050 | 0.0000 | -0.0000
8  3000.0000 | 16.4300 | 0.0076 | -0.0000
9  3500.0000 | 20.2400 | 0.0066
10 4000.0000 | 23.5200
11
12 --- FASE 2: CONSTRUCAO DO POLINOMIO ---
13 Coeficientes de Newton (primeira linha da tabela):
14 [13.93, 0.005, 0.0, -0.0]
15
16 Passo 1: Adicionando 13.9300 * (1.0000)
17 Passo 2: Adicionando 0.0050 * (1.0000x - 2500.0000)
18 Passo 3: Adicionando 0.0000 * (1.0000x^2 - 5500.0000x + 7500000.0000)
19 Passo 4: Adicionando -0.0000 * (1.0000x^3 - 9000.0000x^2 + 26750000.0000x - 26250000000.0000)
20
21 =====
22 RESULTADO FINAL - POLINOMIO INTERPOLADOR UNICO:
23 P(x) = - 0.0000x^3 + 0.0000x^2 - 0.0750x + 85.4800
24 =====

```

Resposta 10.6:

a)

$0,0000x^2 + 0,0014x + 2,6800$ para $x = 1730$ m:

$$0,0000 \cdot 1730^2 + 0,0014 \cdot 1730 + 2,6800 =$$

$$0,0000 \cdot 1730^2 + 0,0014 \cdot 1730 + 2,6800 =$$

$$0 + 2,422 + 2,6800 =$$

5,102 ohms

$- 0,0000x^3 + 0,0000x^2 - 0,0059x + 6,0800$ para $x = 1730$ m:

$$- 0,0000 \cdot 1730^3 + 0,0000 \cdot 1730^2 - 0,0059 \cdot 1730 + 6,0800 =$$

$$0 + 0 - 10,207 + 6,0800 =$$

16,287 ohms

b)

$0,0000x^2 - 0,0094x + 21,0800$ para $x = 3200$:

$$0,0000 \cdot 3200^2 - 0,0094 \cdot 3200 + 21,0800 =$$

$$0 - 30,08 + 21,0800 = - 9 \text{ ohms}$$

- $0,0000x^3 + 0,0000x^2 - 0,0750x + 85,4800$ para $x = 3200$:

- $0,0000 \cdot 3200^3 + 0,0000 \cdot 3200^2 - 0,0750 \cdot 3200 + 85,4800 =$

$0 + 0 - 240 + 85,4800 =$

-154,52 ohms

Problema Teste 6 - Página 318 Questão 10.9

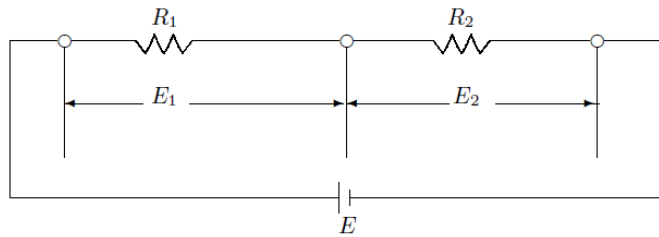
10.9 - A lei de Ohm diz que:

$$E = R I ,$$

onde E é a voltagem, I é a corrente e R a resistência, isto é, o gráfico de $E \times I$ é uma reta de coeficiente angular R que passa pela origem. Vários tipos de resistores, entretanto, não possuem essa propriedade linear. Tal resistor é chamado de um resistor não linear, ou um VARISTOR. Muitos tubos de vácuo são varistores. Geralmente, a relação entre a corrente e a voltagem para um varistor pode ser aproximada por um polinômio da forma:

$$I = a_1 E + a_2 E^2 + \dots + a_n E^n = \sum_{i=1}^n a_i E^i + P_n(E) .$$

Considere agora um circuito consistindo de um resistor linear R_1 e um varistor R_2 , como na figura a seguir:



As equações para o circuito são:

$$I = \frac{E_1}{R_1} = \sum_{i=1}^n a_i E_2^i ;$$

$$E = E_1 + E_2 ,$$

e portanto,

$$R_1 \left(\sum_{i=1}^n a_i E_2^i \right) + E_2 = E \quad \text{ou} \quad R_1 P_n(E_2) + E_2 = E .$$

Suponha que num certo experimento, os seguintes dados foram obtidos:

E_2 (Volts)	0.0	0.5	1.0	1.5	2.0	2.5	3.0	3.5	4.0	4.5
I (Ampères)	0.0	0.0125	0.06	0.195	0.5	1.0875	2.1	3.71	6.12	9.5625

a) Calcule a voltagem total E e a voltagem E_1 quando $E_2 = 2.3$ Volts e $R_1 = 10$ Ohms, usando polinômio de interpolação sobre todos os pontos.

b) Use interpolação inversa de grau 2 para calcular a tensão no varistor quando $E_1 = 10$ Volts e $R_1 = 10$ Ohms.

Entrada 1 – para Interpolação Polinomial de Lagrange

1	0.0	0.0
2	0.5	0.0125
3	1.0	0.06
4	1.5	0.195
5	2.0	0.5
6	2.5	1.0875
7	3.0	2.1
8	3.5	3.71
9	4.0	6.12
10	4.5	9.5625

Entrada 1 – para Interpolação Polinomial de Newton

1	0.0	0.0
2	0.5	0.0125
3	1.0	0.06
4	1.5	0.195
5	2.0	0.5
6	2.5	1.0875
7	3.0	2.1
8	3.5	3.71
9	4.0	6.12
10	4.5	9.5625

Entrada 2 – para Interpolação Polinomial de Lagrange

1	0.5	2.0
2	1.0875	2.5
3	2.1	3.0

Entrada 2 – para Interpolação Polinomial de Newton

1	0.5	2.0
2	1.0875	2.5
3	2.1	3.0

Saída – Interpolação Polinomial de Lagrange (Entrada 1)

```
1  Conjunto de dados lido: ((0.0, 0.0), (0.5, 0.0125),
2  (1.0, 0.06), (1.5, 0.195), (2.0, 0.5), (2.5, 1.0875),
3  (3.0, 2.1), (3.5, 3.71), (4.0, 6.12), (4.5, 9.5625))
4
5  ===== RELATORIO: INTERPOLACAO DE LAGRANGE =====
6
7  --- FASE 1: CONSTRUCAO DOS POLINOMIOS BASE L_i(x) ---
8
9  Calculando L_0(x) para o ponto (0.0, 0.0):
10  Multiplicando por (x - 0.5) / (0.0 - 0.5)
11  Multiplicando por (x - 1.0) / (0.0 - 1.0)
12  Multiplicando por (x - 1.5) / (0.0 - 1.5)
13  Multiplicando por (x - 2.0) / (0.0 - 2.0)
14  Multiplicando por (x - 2.5) / (0.0 - 2.5)
15  Multiplicando por (x - 3.0) / (0.0 - 3.0)
16  Multiplicando por (x - 3.5) / (0.0 - 3.5)
17  Multiplicando por (x - 4.0) / (0.0 - 4.0)
18  Multiplicando por (x - 4.5) / (0.0 - 4.5)
19  L_0(x) resultante = - 0.0014x^9 + 0.0317x^8 - 0.3069x^7 +
20  1.6667x^6 - 5.5796x^5 + 11.8750x^4 - 15.9541x^3 + 12.9266x^2 -
21  5.6579x + 1.0000
22  Adicionando ao polinomio principal: 0.0 * L_0(x)
23
24  Calculando L_1(x) para o ponto (0.5, 0.0125):
25  Multiplicando por (x - 0.0) / (0.5 - 0.0)
26  Multiplicando por (x - 1.0) / (0.5 - 1.0)
27  Multiplicando por (x - 1.5) / (0.5 - 1.5)
28  Multiplicando por (x - 2.0) / (0.5 - 2.0)
29  Multiplicando por (x - 2.5) / (0.5 - 2.5)
30  Multiplicando por (x - 3.0) / (0.5 - 3.0)
31  Multiplicando por (x - 3.5) / (0.5 - 3.5)
32  Multiplicando por (x - 4.0) / (0.5 - 4.0)
33  Multiplicando por (x - 4.5) / (0.5 - 4.5)
34  L_1(x) resultante = 0.0127x^9 - 0.2794x^8 + 2.6222x^7 -
35  13.6889x^6 + 43.3722x^5 - 85.1889x^4 + 100.9929x^3 -
36  65.8429x^2 + 18.0000x
37  Adicionando ao polinomio principal: 0.0125 * L_1(x)
38
```

```

39 Calculando L_2(x) para o ponto (1.0, 0.06):
40     Multiplicando por (x - 0.0) / (1.0 - 0.0)
41     Multiplicando por (x - 0.5) / (1.0 - 0.5)
42     Multiplicando por (x - 1.5) / (1.0 - 1.5)
43     Multiplicando por (x - 2.0) / (1.0 - 2.0)
44     Multiplicando por (x - 2.5) / (1.0 - 2.5)
45     Multiplicando por (x - 3.0) / (1.0 - 3.0)
46     Multiplicando por (x - 3.5) / (1.0 - 3.5)
47     Multiplicando por (x - 4.0) / (1.0 - 4.0)
48     Multiplicando por (x - 4.5) / (1.0 - 4.5)
49     L_2(x) resultante = - 0.0508x^9 + 1.0921x^8 - 9.9556x^7 +
50     50.0444x^6 - 150.8222x^5 + 276.6778x^4 - 297.6714x^3 +
51     167.6857x^2 - 36.0000x
52     Adicionando ao polinomio principal: 0.06 * L_2(x)
53
54 Calculando L_3(x) para o ponto (1.5, 0.195):
55     Multiplicando por (x - 0.0) / (1.5 - 0.0)
56     Multiplicando por (x - 0.5) / (1.5 - 0.5)
57     Multiplicando por (x - 1.0) / (1.5 - 1.0)
58     Multiplicando por (x - 2.0) / (1.5 - 2.0)
59     Multiplicando por (x - 2.5) / (1.5 - 2.5)
60     Multiplicando por (x - 3.0) / (1.5 - 3.0)
61     Multiplicando por (x - 3.5) / (1.5 - 3.5)
62     Multiplicando por (x - 4.0) / (1.5 - 4.0)
63     Multiplicando por (x - 4.5) / (1.5 - 4.5)
64     L_3(x) resultante = 0.1185x^9 - 2.4889x^8 + 22.0444x^7 -
65     106.9333x^6 + 308.2889x^5 - 535.0667x^4 + 537.5481x^3 -
66     279.5111x^2 + 56.0000x
67     Adicionando ao polinomio principal: 0.195 * L_3(x)
68

```

```

69 Calculando L_4(x) para o ponto (2.0, 0.5):
70     Multiplicando por (x - 0.0) / (2.0 - 0.0)
71     Multiplicando por (x - 0.5) / (2.0 - 0.5)
72     Multiplicando por (x - 1.0) / (2.0 - 1.0)
73     Multiplicando por (x - 1.5) / (2.0 - 1.5)
74     Multiplicando por (x - 2.5) / (2.0 - 2.5)
75     Multiplicando por (x - 3.0) / (2.0 - 3.0)
76     Multiplicando por (x - 3.5) / (2.0 - 3.5)
77     Multiplicando por (x - 4.0) / (2.0 - 4.0)
78     Multiplicando por (x - 4.5) / (2.0 - 4.5)
79     L_4(x) resultante = - 0.1778x^9 + 3.6444x^8 -
80     31.3778x^7 + 147.2444x^6 - 408.5444x^5 + 679.1611x^4 -
81     651.9000x^3 + 324.9500x^2 - 63.0000x
82     Adicionando ao polinomio principal: 0.5 * L_4(x)
83
84 Calculando L_5(x) para o ponto (2.5, 1.0875):
85     Multiplicando por (x - 0.0) / (2.5 - 0.0)
86     Multiplicando por (x - 0.5) / (2.5 - 0.5)
87     Multiplicando por (x - 1.0) / (2.5 - 1.0)
88     Multiplicando por (x - 1.5) / (2.5 - 1.5)
89     Multiplicando por (x - 2.0) / (2.5 - 2.0)
90     Multiplicando por (x - 3.0) / (2.5 - 3.0)
91     Multiplicando por (x - 3.5) / (2.5 - 3.5)
92     Multiplicando por (x - 4.0) / (2.5 - 4.0)
93     Multiplicando por (x - 4.5) / (2.5 - 4.5)
94     L_5(x) resultante = 0.1778x^9 - 3.5556x^8 + 29.7778x^7 -
95     135.5556x^6 + 364.1444x^5 - 585.8889x^4 + 545.5000x^3 -
96     265.0000x^2 + 50.4000x
97     Adicionando ao polinomio principal: 1.0875 * L_5(x)
98

```

```

99  √ Calculando L_6(x) para o ponto (3.0, 2.1):
100      Multiplicando por (x - 0.0) / (3.0 - 0.0)
101      Multiplicando por (x - 0.5) / (3.0 - 0.5)
102      Multiplicando por (x - 1.0) / (3.0 - 1.0)
103      Multiplicando por (x - 1.5) / (3.0 - 1.5)
104      Multiplicando por (x - 2.0) / (3.0 - 2.0)
105      Multiplicando por (x - 2.5) / (3.0 - 2.5)
106      Multiplicando por (x - 3.5) / (3.0 - 3.5)
107      Multiplicando por (x - 4.0) / (3.0 - 4.0)
108      Multiplicando por (x - 4.5) / (3.0 - 4.5)
109      L_6(x) resultante = - 0.1185x^9 + 2.3111x^8 - 18.8444x^7 +
110      83.4667x^6 - 218.2889x^5 + 342.6333x^4 - 312.2481x^3 +
111      149.0889x^2 - 28.0000x
112      Adicionando ao polinomio principal: 2.1 * L_6(x)
113
114  √ Calculando L_7(x) para o ponto (3.5, 3.71):
115      Multiplicando por (x - 0.0) / (3.5 - 0.0)
116      Multiplicando por (x - 0.5) / (3.5 - 0.5)
117      Multiplicando por (x - 1.0) / (3.5 - 1.0)
118      Multiplicando por (x - 1.5) / (3.5 - 1.5)
119      Multiplicando por (x - 2.0) / (3.5 - 2.0)
120      Multiplicando por (x - 2.5) / (3.5 - 2.5)
121      Multiplicando por (x - 3.0) / (3.5 - 3.0)
122      Multiplicando por (x - 4.0) / (3.5 - 4.0)
123      Multiplicando por (x - 4.5) / (3.5 - 4.5)
124      L_7(x) resultante = 0.0508x^9 - 0.9651x^8 + 7.6698x^7 -
125      33.1556x^6 + 84.8222x^5 - 130.6222x^4 + 117.1714x^3 -
126      55.2571x^2 + 10.2857x
127      Adicionando ao polinomio principal: 3.71 * L_7(x)
128

```

```

129 Calculando L_8(x) para o ponto (4.0, 6.12):
130     Multiplicando por (x - 0.0) / (4.0 - 0.0)
131     Multiplicando por (x - 0.5) / (4.0 - 0.5)
132     Multiplicando por (x - 1.0) / (4.0 - 1.0)
133     Multiplicando por (x - 1.5) / (4.0 - 1.5)
134     Multiplicando por (x - 2.0) / (4.0 - 2.0)
135     Multiplicando por (x - 2.5) / (4.0 - 2.5)
136     Multiplicando por (x - 3.0) / (4.0 - 3.0)
137     Multiplicando por (x - 3.5) / (4.0 - 3.5)
138     Multiplicando por (x - 4.5) / (4.0 - 4.5)
139     L_8(x) resultante = - 0.0127x^9 + 0.2349x^8 - 1.8222x^7 +
140     7.7111x^6 - 19.3722x^5 + 29.3861x^4 - 26.0429x^3 +
141     12.1679x^2 - 2.2500x
142     Adicionando ao polinomio principal: 6.12 * L_8(x)
143
144 Calculando L_9(x) para o ponto (4.5, 9.5625):
145     Multiplicando por (x - 0.0) / (4.5 - 0.0)
146     Multiplicando por (x - 0.5) / (4.5 - 0.5)
147     Multiplicando por (x - 1.0) / (4.5 - 1.0)
148     Multiplicando por (x - 1.5) / (4.5 - 1.5)
149     Multiplicando por (x - 2.0) / (4.5 - 2.0)
150     Multiplicando por (x - 2.5) / (4.5 - 2.5)
151     Multiplicando por (x - 3.0) / (4.5 - 3.0)
152     Multiplicando por (x - 3.5) / (4.5 - 3.5)
153     Multiplicando por (x - 4.0) / (4.5 - 4.0)
154     L_9(x) resultante = 0.0014x^9 - 0.0254x^8 + 0.1926x^7 -
155     0.8000x^6 + 1.9796x^5 - 2.9667x^4 + 2.6041x^3 - 1.2079x^2 +
156     0.2222x
157     Adicionando ao polinomio principal: 9.5625 * L_9(x)
158
159 =====
160 RESULTADO FINAL - POLINOMIO INTERPOLADOR UNICO:
161 P(x) = 0.0200x^4 + 0.0100x^3 + 0.0200x^2 + 0.0100x
162 =====

```

Saída – Interpolação Polinomial de Newton (Entrada 1)

```

1  Conjunto de dados lido: ((0.0, 0.0), (0.5, 0.0125), (1.0, 0.06), (1.5, 0.195),
2  (2.0, 0.5), (2.5, 1.0875), (3.0, 2.1), (3.5, 3.71), (4.0, 6.12), (4.5, 9.5625))
3
4  ===== RELATORIO: INTERPOLACAO DE NEWTON =====
5
6  --- FASE 1: TABELA DE DIFERENCAS DIVIDIDAS ---
7  x      | Ordem 0 | Ordem 1 | ...
8  0.0000 | 0.0000 | 0.0250 | 0.0700 | 0.0700 | 0.0200 | -0.0000 | 0.0000 | -0.0000 | 0.0000 | -0.0000
9  0.5000 | 0.0125 | 0.0950 | 0.1750 | 0.1100 | 0.0200 | 0.0000 | -0.0000 | 0.0000 | -0.0000
10 1.0000 | 0.0600 | 0.2700 | 0.3400 | 0.1500 | 0.0200 | -0.0000 | 0.0000 | -0.0000
11 1.5000 | 0.1950 | 0.6100 | 0.5650 | 0.1900 | 0.0200 | 0.0000 | -0.0000
12 2.0000 | 0.5000 | 1.1750 | 0.8500 | 0.2300 | 0.0200 | -0.0000
13 2.5000 | 1.0875 | 2.0250 | 1.1950 | 0.2700 | 0.0200
14 3.0000 | 2.1000 | 3.2200 | 1.6000 | 0.3100
15 3.5000 | 3.7100 | 4.8200 | 2.0650
16 4.0000 | 6.1200 | 6.8850
17 4.5000 | 9.5625
18

```

```

19 --- FASE 2: CONSTRUCAO DO POLINOMIO ---
20 Coeficientes de Newton (primeira linha da tabela):
21 [0.0, 0.025, 0.07, 0.07, 0.02, -0.0, 0.0, -0.0, 0.0, -0.0]
22
23 Passo 1: Adicionando 0.0000 * (1.0000)
24 Passo 2: Adicionando 0.0250 * (1.0000x)
25 Passo 3: Adicionando 0.0700 * (1.0000x^2 - 0.5000x)
26 Passo 4: Adicionando 0.0700 * (1.0000x^3 - 1.5000x^2 + 0.5000x)
27 Passo 5: Adicionando 0.0200 * (1.0000x^4 - 3.0000x^3 + 2.7500x^2 - 0.7500x)
28 Passo 6: Adicionando -0.0000 * (1.0000x^5 - 5.0000x^4 + 8.7500x^3 - 6.2500x^2 + 1.5000x)
29 Passo 7: Adicionando 0.0000 * (1.0000x^6 - 7.5000x^5 + 21.2500x^4 - 28.1250x^3 + 17.1250x^2 - 3.7500x)
30 Passo 8: Adicionando -0.0000 * (1.0000x^7 - 10.5000x^6 + 43.7500x^5 - 91.8750x^4 + 101.5000x^3 - 55.1250x^2 + 11.2500x)
31 Passo 9: Adicionando 0.0000 * (1.0000x^8 - 14.0000x^7 + 80.5000x^6 - 245.0000x^5 + 423.0625x^4 - 410.3750x^3 + 204.1875x^2 - 39.3750x)
32 Passo 10: Adicionando -0.0000 * (1.0000x^9 - 18.0000x^8 + 136.5000x^7 - 567.0000x^6 + 1403.0625x^5 - 2102.6250x^4 + 1845.6875x^3 - 856.1250x^2 + 157.5000x)
33
34 =====
35 RESULTADO FINAL - POLINOMIO INTERPOLADOR UNICO:
36 P(x) = 0.0200x^4 + 0.0100x^3 + 0.0200x^2 + 0.0100x
37 =====

```


Saída - Interpolação Polinomial de Lagrange (Entrada 2)

```
1  Conjunto de dados lido: ((0.5, 2.0), (1.0875, 2.5), (2.1, 3.0))
2
3  ===== RELATORIO: INTERPOLACAO DE LAGRANGE =====
4
5  --- FASE 1: CONSTRUCAO DOS POLINOMIOS BASE L_i(x) ---
6
7  √ Calculando L_0(x) para o ponto (0.5, 2.0):
8      Multiplicando por (x - 1.0875) / (0.5 - 1.0875)
9      Multiplicando por (x - 2.1) / (0.5 - 2.1)
10     L_0(x) resultante = 1.0638x^2 - 3.3910x + 2.4295
11     Adicionando ao polinomio principal: 2.0 * L_0(x)
12
13  √ Calculando L_1(x) para o ponto (1.0875, 2.5):
14     Multiplicando por (x - 0.5) / (1.0875 - 0.5)
15     Multiplicando por (x - 2.1) / (1.0875 - 2.1)
16     L_1(x) resultante = - 1.6811x^2 + 4.3709x - 1.7652
17     Adicionando ao polinomio principal: 2.5 * L_1(x)
18
19  √ Calculando L_2(x) para o ponto (2.1, 3.0):
20     Multiplicando por (x - 0.5) / (2.1 - 0.5)
21     Multiplicando por (x - 1.0875) / (2.1 - 1.0875)
22     L_2(x) resultante = 0.6173x^2 - 0.9799x + 0.3356
23     Adicionando ao polinomio principal: 3.0 * L_2(x)
24
25  =====
26  RESULTADO FINAL - POLINOMIO INTERPOLADOR UNICO:
27  P(x) = - 0.2233x^2 + 1.2055x + 1.4531
28  =====
```

Saída – Interpolação Polinomial de Newton (Entrada 2)

```
1  Conjunto de dados lido: ((0.5, 2.0), (1.0875, 2.5), (2.1, 3.0))
2
3  ===== RELATORIO: INTERPOLACAO DE NEWTON =====
4
5  --- FASE 1: TABELA DE DIFERENCAS DIVIDIDAS ---
6  x          | Ordem 0   | Ordem 1   | ...
7  0.5000     | 2.0000    | 0.8511    | -0.2233
8  1.0875     | 2.5000    | 0.4938
9  2.1000     | 3.0000
10
11 --- FASE 2: CONSTRUCAO DO POLINOMIO ---
12 Coeficientes de Newton (primeira linha da tabela):
13 [2.0, 0.8511, -0.2233]
14
15 Passo 1: Adicionando 2.0000 * (1.0000)
16 Passo 2: Adicionando 0.8511 * (1.0000x - 0.5000)
17 Passo 3: Adicionando -0.2233 * (1.0000x^2 - 1.5875x + 0.5437)
18
19 =====
20 RESULTADO FINAL - POLINOMIO INTERPOLADOR UNICO:
21  $P(x) = -0.2233x^2 + 1.2055x + 1.4531$ 
22 =====
```

Resposta 10.9:

a)

$0,0200x^4 + 0,0100x^3 + 0,0200x^2 + 0,0100x$ para $x = 2,3$:

$$I = 0,0200 \cdot (2,3)^4 + 0,0100 \cdot (2,3)^3 + 0,0200 \cdot (2,3)^2 + 0,0100 \cdot (2,3)$$

$$\mathbf{I = 0,810152 \text{ A}}$$

$$E_1 = 0,810152 \cdot 10$$

$$\mathbf{E_1 = 8,10152 \text{ V}}$$

$$E = 8,10152 + 2,3$$

$$\mathbf{E = 10,40152 \text{ V}}$$

b)

$-0,2233x^2 + 1,2055x + 1,4531$ para $x = 1$:

$$E_2 = -0,2233 \cdot 1^2 + 1,2055 \cdot 1 + 1,4531$$

$$E_2 = -0,2233 + 1,2055 + 1,4531$$

$$\mathbf{E_2 = 2,4353 \text{ V}}$$

Derivação Numérica de Primeira Ordem

Estratégia de Implementação:

A implementação da derivada de primeira ordem foi baseada no método das Diferenças Finitas Centrais. A estratégia consistiu em avaliar a função em pontos vizinhos simétricos ($x_0 + h$ e $x_0 - h$) usando um passo h predefinido de 10^{-5} , um valor que oferece um excelente equilíbrio computacional. A fórmula $f'(x) = [f(x_0 + h) - f(x_0 - h)] / (2h)$ foi aplicada de forma direta. Para permitir a leitura de funções dinâmicas, empregou-se a função nativa `eval` do Python dentro de um ambiente seguro previamente mapeado com as operações da biblioteca `math`.

Estrutura dos Arquivos de Entrada/Saída

A estrutura do arquivo de entrada (`entrada.txt`) foi modificada para esta classe de problemas. Como a derivação ocorre em um ponto específico de uma função contínua, cada linha recebe a expressão matemática e o valor numérico de x_0 , separados por um ponto e vírgula. O arquivo de saída (`saida.txt`) manteve seu papel de memorial de cálculo, imprimindo os valores exatos encontrados para $f(x_0 + h)$ e $f(x_0 - h)$, além de exibir a fórmula preenchida antes de revelar a taxa de variação final.

Dificuldades enfrentadas

A principal dificuldade enfrentada nesta implementação foi o fenômeno do cancelamento catastrófico (erros de arredondamento em subtrações de números muito próximos). A escolha do passo h precisou ser milimétrica; valores menores que 10^{-5} começavam a introduzir problemas de ponto flutuante que corrompiam a precisão da derivada. Além disso, garantir que o interpretador traduzisse corretamente as strings matemáticas do usuário (como `sin(x)` em vez de `math.sin(x)`) exigiu a construção de um dicionário de atalhos para que a leitura não falhasse durante a execução.

Derivação Numérica de Segunda Ordem

Estratégia de Implementação:

A implementação da derivada de segunda ordem seguiu o método das Diferenças Finitas Centrais, cujo objetivo geométrico é medir a concavidade da curva. A estratégia consistiu em avaliar a expressão matemática em três pontos distintos (o ponto central x_0 , e seus vizinhos simétricos $x_0 + h$ e $x_0 - h$), aplicando a fórmula $f''(x) = [f(x_0 + h) - 2f(x_0) + f(x_0 - h)] / (h^2)$. Foi mantido o passo h fixo de 10^{-5} e reaproveitado o mesmo ambiente seguro de avaliação dinâmica (eval) construído no método anterior, garantindo consistência na leitura das strings matemáticas.

Estrutura dos Arquivos de Entrada/Saída

A estrutura do arquivo de entrada (entrada.txt) permaneceu idêntica à da derivação de primeira ordem, recebendo a função e o ponto numérico de avaliação separados por um ponto e vírgula. O arquivo de saída (saida.txt) continuou atuando como um memorial de cálculo, mas foi expandido para registrar a avaliação dos três pontos de interesse. Uma adaptação importante de formatação foi programar o código para exibir o divisor h^2 em notação científica diretamente no documento de texto, evitando a poluição visual que uma dezena de casas decimais nulas causaria na leitura da fórmula.

Dificuldades enfrentadas

A dificuldade mais crítica nesta implementação foi lidar com a severa amplificação de erros computacionais. Como a fórmula da segunda derivada exige a divisão por h ao quadrado (resultando em um número minúsculo na grandeza de 10^{-10}), qualquer mínima imprecisão de truncamento durante a subtração no numerador sofre o que chamamos de cancelamento catastrófico, inflando o erro da resposta final. A escolha de manter o passo h em 10^{-5} foi o limite encontrado para balancear a precisão teórica do limite matemático sem colapsar a aritmética de ponto flutuante do interpretador Python.

Integração por Trapézios Simples e Compostos

Estratégia de Implementação:

A implementação da Regra dos Trapézios unificou as abordagens simples e múltipla (composta) em uma única estrutura lógica, otimizando o código. A estratégia baseou-se na verificação do parâmetro de subintervalos n fornecido. Se o valor for 1, o algoritmo aplica a fórmula do trapézio simples, calculando a área apenas com os limites extremos. Se n for maior que 1, o sistema aciona automaticamente a regra múltipla, gerando o vetor de pontos intermediários e aplicando o peso correto (multiplicação por 2) aos termos internos. O comando `eval` continuou sendo utilizado no ambiente seguro para calcular a função dinamicamente.

Estrutura dos Arquivos de Entrada/Saída

O arquivo de entrada (`entrada.txt`) recebeu um novo formato padrão para atender à integração. Cada linha foi estruturada para conter a expressão matemática, os limites a e b , e o número de subintervalos n , todos separados por ponto e vírgula (exemplo: `sin(x) ; 0, 3.14 ; 4`). O arquivo de saída (`saida.txt`) age como um relatório detalhado: ele imprime os vetores de x e $f(x)$ utilizando a formatação com parênteses, avisa qual das versões da fórmula foi ativada e exibe o somatório isolado dos termos antes da exibição da área final.

Dificuldades enfrentadas

A principal dificuldade enfrentada foi o processamento do arquivo de entrada, já que a string precisava ser dividida e convertida em três tipos de dados diferentes simultaneamente (texto para a função, floats para os limites e int para o passo). No aspecto matemático, o desafio foi gerenciar corretamente as listas em Python durante a regra múltipla; foi necessário aplicar fatiamentos (slicing) precisos para separar o primeiro e o último termo da função, garantindo que os limites não fossem dobrados por engano durante a soma dos termos internos.

Integração por Simpson 1/3

Estratégia de Implementação:

A implementação da Regra de Simpson 1/3 (versão múltipla) foi desenvolvida para calcular a área sob a curva com maior precisão do que o método dos trapézios, utilizando aproximações quadráticas (arcos de parábola). A estratégia consistiu em subdividir o intervalo e aplicar os pesos matemáticos característicos do método: os limites extremos mantêm peso 1, os pontos internos de índice ímpar são multiplicados por 4, e os de índice par são multiplicados por 2. As expressões continuaram sendo avaliadas dinamicamente pelo comando `eval` dentro do ambiente seguro.

Estrutura dos Arquivos de Entrada/Saída

A estrutura do arquivo de entrada (`entrada.txt`) segue o padrão unificado da integração, onde cada linha contém a função, os limites a e b , e o parâmetro de subintervalos n , separados por ponto e vírgula. O arquivo de saída (`saida.txt`) funciona como um memorial analítico: ele lista os vetores de pontos formatados com parênteses e exibe isoladamente o valor das somatórias dos índices pares e ímpares, facilitando muito a correção manual e a compreensão de como a fórmula foi montada pelo algoritmo.

Dificuldades enfrentadas

A principal dificuldade técnica enfrentada nesta etapa foi a restrição estrutural do próprio método, que exige obrigatoriamente um número par de subintervalos para funcionar corretamente. Executar a fórmula com um n ímpar geraria um falso resultado matemático sem que a linguagem Python acusasse erro nativo. Para resolver esse risco silencioso, foi necessário criar uma trava de segurança que verifica o parâmetro lido antes de iniciar as contas; caso n seja ímpar, a função é abortada e um alerta de erro fatal é gravado no relatório. Além disso, separar corretamente as listas em índices pares e ímpares exigiu fatiamentos (slicing) e laços de repetição muito bem calibrados.

Integração por Simpson 3/8

Estratégia de Implementação:

A implementação da Regra de Simpson 3/8 múltipla foi desenhada para obter alta precisão na integração utilizando aproximações por polinômios cúbicos. A estratégia computacional consistiu em subdividir o intervalo e aplicar uma distribuição rígida de pesos aos pontos: os limites extremos recebem peso 1, os pontos com índices múltiplos de 3 são multiplicados por 2, e todos os demais pontos internos recebem peso 3. A avaliação dinâmica das funções matemáticas continuou isolada no ambiente seguro gerido pelo comando eval do Python.

Estrutura dos Arquivos de Entrada/Saída

O arquivo de entrada (entrada.txt) manteve a mesma estrutura padronizada da seção de integração, recebendo a função, os limites do intervalo e o número de subintervalos n , separados por ponto e vírgula. O arquivo de saída (saida.txt) opera como um memorial descritivo completo. Ele apresenta os vetores de dados formatados com parênteses e registra detalhadamente o somatório isolado dos valores que são e não são múltiplos de 3, demonstrando com clareza como o algoritmo construiu a resposta final.

Dificuldades enfrentadas

A principal dificuldade enfrentada nesta implementação foi a gestão lógica das condições do método. A regra de Simpson 3/8 exige estritamente que o número de subintervalos n seja um múltiplo de 3. Para impedir que cálculos inválidos fossem processados de forma silenciosa, implementei uma validação inicial que aborta a função e emite um alerta de erro caso o usuário forneça um valor incompatível. O segundo desafio foi a separação correta dos pontos dentro dos laços de repetição, o que exigiu o uso milimétrico do operador de resto de divisão (módulo) para garantir que os multiplicadores 2 e 3 fossem aplicados às coordenadas certas.

Integração com Extrapolação de Richardson

Estratégia de Implementação:

A Extrapolação de Richardson foi implementada como uma técnica de aceleração de convergência. A estratégia consistiu em calcular a integral de uma mesma função duas vezes utilizando a Regra dos Trapézios: a primeira com uma malha base de subintervalos (n_1) e a segunda com a malha duplicada ($n_2 = 2 * n_1$). Com esses dois valores em mãos, o algoritmo aplica a fórmula algébrica de correção $R = I_2 + (I_2 - I_1) / 3$, que cancela o termo principal do erro de truncamento. As funções continuaram sendo avaliadas pelo interpretador dinâmico do Python.

Estrutura dos Arquivos de Entrada/Saída

O arquivo de entrada (entrada.txt) utiliza a mesma estrutura unificada: a expressão matemática, os limites do intervalo e o parâmetro base n_1 , todos separados por ponto e vírgula. O arquivo de saída (saida.txt) foi cuidadosamente planejado para ser direto: ele omite os cálculos repetitivos da área e registra apenas os dois valores integrais base encontrados (I_1 e I_2), exibindo em seguida a substituição desses valores na fórmula de extrapolação até chegar ao resultado final.

Dificuldades enfrentadas

A principal dificuldade nesta implementação foi o controle do fluxo de gravação no arquivo de saída (log). Se o algoritmo chamasse a função padrão da Regra dos Trapézios duas vezes, o relatório seria poluído com a impressão de dezenas de pontos intermediários, tornando a leitura confusa. A solução arquitetural para esse problema foi criar uma função auxiliar oculta (chamada internamente de trapézio silencioso). Essa função realiza toda a matemática pesada na memória e devolve apenas o número final para o método de Richardson, garantindo que o relatório documente apenas a essência da extrapolação.

Integração com Quadratura de Gauss

Estratégia de Implementação:

A implementação da Quadratura de Gauss-Legendre foi desenhada para atingir altíssima precisão com poucas avaliações da função, abandonando a ideia de pontos igualmente espaçados. A estratégia principal consistiu em programar o mapeamento linear que converte o intervalo original da integral para o intervalo padrão de -1 a 1. Para manter o código leve e evitar cálculos complexos de raízes de polinômios em tempo de execução, optei por criar um dicionário interno armazenando as raízes (t) e os pesos (w) exatos para os casos de 2, 3 e 4 pontos. O algoritmo faz a mudança de variável, avalia os pontos ótimos e soma os resultados ponderados.

Estrutura dos Arquivos de Entrada/Saída

O arquivo de entrada (`entrada.txt`) segue rigorosamente a padronização estabelecida, lendo a expressão matemática, os limites do intervalo e o parâmetro numérico n , separados por ponto e vírgula. A grande diferença funcional é que, neste método, n representa a escolha da quantidade de pontos ótimos da tabela de Legendre. O arquivo de saída (`saida.txt`) age como um relatório detalhado: ele registra a dedução da mudança de variável (o cálculo do dx) e imprime uma lista ponto a ponto com os valores de t , w e $f(x)$ utilizados, demonstrando a montagem final do produto escalar.

Dificuldades enfrentadas

A principal dificuldade técnica desta implementação foi o bloqueio de limites do método. Como a tabela de pesos e raízes foi pré-configurada apenas para 2, 3 ou 4 pontos, o programa corria o risco de quebrar (`index out of range`) se o usuário pedisse um valor diferente. Para garantir a robustez, construí uma validação rígida que intercepta números fora desse escopo e aborta a execução com um alerta claro no log. Além disso, a aplicação da fórmula de mudança de variável exigiu cuidado extremo com o isolamento algébrico no código Python para garantir que o mapeamento do ponto x fosse perfeito antes da função `eval` processá-lo.

Problema Teste 7 - Página 366 Questão 11.1

11.1 - Um corpo negro (radiador perfeito) emite energia em uma taxa proporcional à quarta potência de sua temperatura absoluta, de acordo com a equação de Stefan-Boltzmann,

$$E = 36.9 \cdot 10^{-12} T^4 ,$$

onde E = potência de emissão, W/cm^2 e T = temperatura, K° .

O que se deseja é determinar uma fração dessa energia contida no espectro visível, que é tomado aqui como sendo 4.10^{-5} a 7.10^{-5} cm. Podemos obter a parte visível integrando a equação de Planck entre esses limites:

$$E_{visível} = \int_{4.10^{-5}}^{7.10^{-5}} \frac{2.39 \cdot 10^{-11}}{x^5 (e^{1.432/Tx} - 1)} dx ,$$

onde x = comprimento de onda, cm; E e T como definido acima.

A eficiência luminosa é definida como a relação da energia no espectro visível para a energia total. Se multiplicarmos por 100 para obter a eficiência percentual e combinarmos as constantes, o problema torna-se o de calcular:

$$EFF = \left(64.77 \int_{4.10^{-5}}^{7.10^{-5}} \frac{dx}{x^5 (e^{1.432/Tx} - 1)} \right) / T^4 .$$

Obter a eficiência luminosa, com erro relativo $< 10^{-5}$, nas seguintes condições:

i) $T_i = 2000^\circ K$
 $T_f = 3000^\circ K$

com incremento da temperatura igual a 250.

ii) $T_i = 2000^\circ K$
 $T_f = 3000^\circ K$

com incremento da temperatura igual a 200.

onde T_i e T_f são as temperaturas iniciais e finais, respectivamente.

Entrada 1 - para Integração com Simpson 1/3

```
1  1 / (x**5 * (exp(1.432 / (2000 * x)) - 1)) ; 4e-5, 7e-5 ; 1000
2  1 / (x**5 * (exp(1.432 / (2250 * x)) - 1)) ; 4e-5, 7e-5 ; 1000
3  1 / (x**5 * (exp(1.432 / (2500 * x)) - 1)) ; 4e-5, 7e-5 ; 1000
4  1 / (x**5 * (exp(1.432 / (2750 * x)) - 1)) ; 4e-5, 7e-5 ; 1000
5  1 / (x**5 * (exp(1.432 / (3000 * x)) - 1)) ; 4e-5, 7e-5 ; 1000
```

Saída – Integração com Simpson 1/3 (Entrada 1)

```
1  ===== RELATORIO DE PROCESSAMENTO =====
2
3  =====
4  EQUACAO ANALISADA 1: f(x) = 1 / (x**5 * (exp(1.432 / (2000 * x)) - 1))
5  Intervalo de integracao: [4e-05, 7e-05]
6  =====
7
8  Metodo: Regra de Simpson 1/3 | Subintervalos (n) = 1000
9  Passo (h) = 0.000000
10
11 --- FASE 1: AVALIACAO DOS PONTOS ---
12 Vetor de pontos x = (Pontos)
13
14 --- FASE 2: APLICACAO DA FORMULA ---
15 Formula: I = (h / 3) * (f(x0) + 4*soma(impares) + 2*soma(pares) + f(xn))
16 Soma indices impares (Peso 4) = 3317609854562985984.000000
17 Soma indices pares (Peso 2) = 3306792408719675392.000000
18 Calculo executado: (0.000000 / 3) * [19905682415772454912.000000]
19
20 -----
21 VALOR DA INTEGRAL ENCONTRADO: I = 199056824157.724487
22 -----
23
24 =====
25 EQUACAO ANALISADA 2: f(x) = 1 / (x**5 * (exp(1.432 / (2250 * x)) - 1))
26 Intervalo de integracao: [4e-05, 7e-05]
27 =====
28
29 Metodo: Regra de Simpson 1/3 | Subintervalos (n) = 1000
30 Passo (h) = 0.000000
31
32 --- FASE 1: AVALIACAO DOS PONTOS ---
33 Vetor de pontos x = (Pontos)
34
```

OBS.: todos os pontos foram substituídos pela palavra “Pontos”, porque o arquivo de saída ficou extremamente longo por conta deles, e seria inviável mostrar todos aqui.

Problema Teste 8 - Página 368 Questão 11.6

11.6 - Na determinação da radiação luminosa emitida por um radiador perfeito é necessário calcular-se o valor da integral:

$$Q = \int_{\lambda_1}^{\lambda_2} \frac{2\pi hc^2}{\lambda^5 (e^{\frac{hc}{k\lambda T}} - 1)} d\lambda,$$

onde:

Q = radiação emitida por unidade de tempo por unidade de área entre os comprimentos de onda λ_1 e λ_2 , em $\text{erg/cm}^2 \cdot \text{s}$,

λ_1 e λ_2 = limites inferior e superior, respectivamente, do comprimento de onda, em cm ,

h = constante de Planck = $6.6256 \times 10^{-27} \text{ erg.s}$,

c = velocidade da luz = $2.99793 \times 10^{10} \text{ cm/s}$,

k = constante de Boltzmann = $1.38054 \times 10^{-16} \text{ erg/k}$,

T = temperatura absoluta da superfície, $^{\circ}\text{K}$,

λ = variável de integração = comprimento de onda, cm .

Obter Q , com erro relativo $< 10^{-5}$, nas seguintes condições:

i) $\lambda_1 = 3.933666 \times 10^{-5} \text{ cm}$,
 $\lambda_2 = 5.895923 \times 10^{-5} \text{ cm}$,
 $T = 2000 \text{ }^{\circ}\text{K}$.

ii) $\lambda_1 = 3.933666 \times 10^{-5} \text{ cm}$,
 $\lambda_2 = 5.895923 \times 10^{-5} \text{ cm}$,
 $T = 6000 \text{ }^{\circ}\text{K}$.

Entrada 1 - para Integração com Simpson 1/3

```
1 (2 * pi * 6.6256e-27 * (2.99793e10)**2) / (x**5 * (exp((6.6256e-27 * 2.99793e10) / (1.38054e-16 * x * 2000)) - 1)) ; 3.933666e-5, 5.895923e-5 ;  
1000  
2 (2 * pi * 6.6256e-27 * (2.99793e10)**2) / (x**5 * (exp((6.6256e-27 * 2.99793e10) / (1.38054e-16 * x * 6000)) - 1)) ; 3.933666e-5, 5.895923e-5 ;  
1000
```

Saída – Integração com Simpson 1/3 (Entrada 1)

```
1  ===== RELATORIO DE PROCESSAMENTO =====
2
3  =====
4  EQUACAO ANALISADA 1: f(x) = (2 * pi * 6.6256e-27 * (2.99793e10)**2) /
5  (x**5 * (exp((6.6256e-27 * 2.99793e10) / (1.38054e-16 * x * 2000)) - 1))
6  Intervalo de integracao: [3.933666e-05, 5.895923e-05]
7  =====
8
9  Metodo: Regra de Simpson 1/3 | Subintervalos (n) = 1000
10 Passo (h) = 0.000000
11
12 --- FASE 1: AVALIACAO DOS PONTOS ---
13 Vetor de pontos x = (Pontos)
14
15 --- FASE 2: APLICACAO DA FORMULA ---
16 Formula: I = (h / 3) * (f(x0) + 4*soma(impares) + 2*soma(pares) + f(xn))
17 Soma indices impares (Peso 4) = 41582922430625.539062
18 Soma indices pares (Peso 2) = 41448923647250.070312
19 Calculo executado: (0.000000 / 3) * [249497835648715.406250]
20
21 -----
22 VALOR DA INTEGRAL ENCONTRADO: I = 1631929.581622
23 -----
24
25 =====
26 EQUACAO ANALISADA 2: f(x) = (2 * pi * 6.6256e-27 * (2.99793e10)**2) /
27 (x**5 * (exp((6.6256e-27 * 2.99793e10) / (1.38054e-16 * x * 6000)) - 1))
28 Intervalo de integracao: [3.933666e-05, 5.895923e-05]
29 =====
30
31 Metodo: Regra de Simpson 1/3 | Subintervalos (n) = 1000
32 Passo (h) = 0.000000
33
34 --- FASE 1: AVALIACAO DOS PONTOS ---
35 Vetor de pontos x = (Pontos)
36
37 --- FASE 2: APLICACAO DA FORMULA ---
38 Formula: I = (h / 3) * (f(x0) + 4*soma(impares) + 2*soma(pares) + f(xn))
39 Soma indices impares (Peso 4) = 484271589163594944.000000
40 Soma indices pares (Peso 2) = 483365632081140480.000000
41 Calculo executado: (0.000000 / 3) * [2905629155574481408.000000]
42
43 -----
44 VALOR DA INTEGRAL ENCONTRADO: I = 19005303833.100380
45 -----
```

OBS.: todos os pontos foram substituídos pela palavra “Pontos”, porque o arquivo de saída ficou extremamente longo por conta deles, e seria inviável mostrar todos aqui.

Problema Teste 9 - Página 371 Questão 11.11

11.11 - O serviço de proteção ao consumidor (SPC) tem recebido muitas reclamações quanto ao peso real do pacote de 5kg do açúcar vendido nos supermercados. Para verificar a validade das reclamações, o SPC contratou uma firma especializada em estatística para fazer uma estimativa da quantidade de pacotes que realmente continham menos de 5kg. Como é inviável a rezeagem de todos os pacotes, a firma responsável pesou apenas uma amostra de 100 pacotes. A partir destes dados e utilizando métodos estatísticos eles puderam ter uma boa idéia do peso de todos os pacotes existentes no mercado.

Chamando de x_i o peso do pacote i , tem-se que a média da amostra $\bar{x}$ é dada por:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i ,$$

onde n é o número de pacotes da amostra. Serão omitidos os pesos, face ao elevado número de pacotes examinados.

Calculando-se a média:

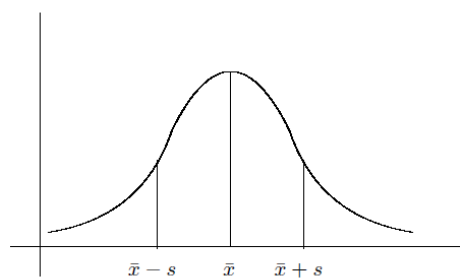
$$\bar{x} = \frac{1}{100} \times 499.1 = 4.991kg .$$

O desvio padrão que é uma medida estatística que dá uma noção da dispersão dos pesos em relação à média é dado por:

$$S = + \sqrt{\frac{1}{n-1} \sum_{i=1}^n (\bar{x} - x_i)^2} .$$

Para os dados deste problema tem-se que $S = 0.005kg$.

Supondo-se verdadeira a hipótese de que a variação do peso dos pacotes não é tendenciosa, isto é, que o peso de um pacote é função de uma composição de efeitos de outras variáveis independentes, entre as quais podemos citar: regulagem da máquina de ensacar, variação da densidade do açúcar, leitura do peso, etc.; pode-se afirmar que a variável do peso tem uma distribuição normal. O gráfico da distribuição normal é apresentado a seguir:



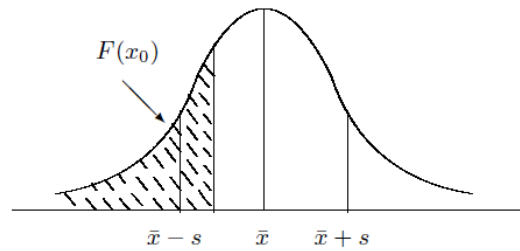
A forma analítica desta função é:

$$f(x) = \frac{1}{S\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\bar{x}}{S}\right)^2} .$$

O valor de $f(x)$ é a frequência de ocorrência do valor x . A integral de $f(x)$ fornece a frequência acumulada, isto é:

$$F(x_0) = \int_{-\infty}^{x_0} f(x) dx ,$$

é a probabilidade de que x assumo um valor menor ou igual a x_0 . Graficamente, $F(x_0)$ é a área hachurada na figura a seguir:



No problema em questão, o que se deseja é determinar :

$$F(5) = \int_{-\infty}^5 \frac{1}{0.005 \sqrt{2\pi}} e^{\frac{1}{2} \left(\frac{x-4.991}{0.005} \right)^2} dx .$$

Observações:

1) $\int_{-\infty}^{\infty} f(x) dx = 1.$

2) A curva é simétrica em relação a média ($\bar{x}$), logo:

$$\int_{-\infty}^{\bar{x}} f(x) dx = \int_{\bar{x}}^{\infty} f(x) dx = 0.5.$$

Entrada 1 – para Integração com Extrapolação de Richardson

```
1 (1 / (0.005 * sqrt(2 * pi))) * exp(-0.5 * ((x - 4.991) / 0.005)**2) ; 4.991, 5.0 ; 1000
```

Saída – Integração com Extrapolação de Richardson (Entrada 1)

```
1 ===== RELATORIO DE PROCESSAMENTO =====
2
3 =====
4 EQUACAO ANALISADA 1: f(x) = (1 / (0.005 * sqrt(2 * pi))) * exp(-0.5 * ((x - 4.991) / 0.005)**2)
5 Intervalo de integracao: [4.991, 5.0]
6 =====
7
8 Metodo: Extrapolacao de Richardson
9 Estrategia: Combinar duas malhas de Trapezio (n1=1000 e n2=2000)
10
11 --- FASE 1: OBTENCAO DOS CALCULOS BASE ---
12 I(h1) [Integral obtida com n=1000] = 0.464070
13 I(h2) [Integral obtida com n=2000] = 0.464070
14
15 --- FASE 2: EXTRAPOLACAO ALGEBRICA ---
16 Formula: R = I(h2) + [I(h2) - I(h1)] / 3
17 Termo de correcao estimado = 0.000000
18 Calculo final: 0.464070 + (0.464070 - 0.464070) / 3
19
20 -----
21 VALOR DA INTEGRAL ENCONTRADO: I = 0.464070
22 -----
```


Resposta 11.11:

a) Somando com a metade esquerda do gráfico temos: $0,464070 + 0,5 = 0,96407$, ou seja:

96,407%

Considerações Finais

O presente relatório documentou a implementação, o teste e a análise de métodos numéricos clássicos focados em duas frentes fundamentais da computação científica: o ajuste de curvas para modelagem de dados e o cálculo numérico avançado. Através da linguagem Python, foi possível estruturar um laboratório de testes modular e automatizado, separando a lógica matemática em arquivos de entrada e saída. O uso de recursos nativos, como o tratamento cuidadoso de strings e a função de avaliação dinâmica em ambientes seguros, permitiu generalizar a leitura de conjuntos de dados e equações complexas em tempo de execução.

Na primeira etapa do projeto, dedicada ao tratamento de dados, a transição entre os métodos implementados evidenciou o contraste conceitual entre tendência e exatidão. A Aproximação por Mínimos Quadrados (nas abordagens linear, polinomial discreta e contínua) provou ser a ferramenta ideal para capturar o comportamento global de um conjunto de valores. Ao tolerar desvios exatos em prol de uma curva suave, o MMQ mostra-se essencial para lidar com dados experimentais reais, que naturalmente contêm ruídos. Em contrapartida, os métodos de Interpolação (Lagrange e Diferenças Divididas de Newton) demonstraram sua força onde a precisão absoluta é exigida. Ao forçar o polinômio a passar exatamente por cada ponto fornecido, a interpolação torna-se a escolha matemática perfeita para preencher lacunas em tabelas de dados altamente confiáveis.

Na segunda etapa, voltada ao cálculo diferencial e integral, a implementação expôs os limites práticos da aritmética computacional da máquina. A Derivação Numérica de primeira e segunda ordem por diferenças finitas centrais escancarou o dilema do erro de truncamento versus o cancelamento catastrófico: buscar a precisão teórica com um passo excessivamente pequeno inevitavelmente corrompe a resposta devido às limitações do ponto flutuante. Já na Integração Numérica, observou-se uma fascinante evolução algorítmica. As regras de Newton-Cotes (Trapézios e Simpson) forneceram uma base sólida e direta. Contudo, foram as abordagens avançadas que demonstraram o verdadeiro poder da análise numérica: a

Extrapolação de Richardson combinou cálculos simples para anular o erro de forma algébrica, enquanto a Quadratura de Gauss alcançou precisões altas com pouquíssimas avaliações da função ao otimizar inteligentemente os pontos de amostragem no gráfico.

Além da construção dos algoritmos em si, o verdadeiro marco de maturidade técnica deste trabalho residiu na resolução dos exercícios práticos. Esta etapa exigiu um raciocínio analítico rigoroso para interpretar enunciados físicos e de engenharia abstratos, extrair as funções contínuas ou amostragens ocultas no texto, e traduzi-las em modelos matemáticos solucionáveis. A capacidade de olhar para um problema prático e decidir estrategicamente se ele demanda uma regressão para prever o futuro, ou uma derivada para encontrar uma taxa instantânea de variação, representa a ponte definitiva entre a teoria e a modelagem do mundo físico.

Em suma, o desenvolvimento e a análise deste vasto conjunto de algoritmos consolidaram a base teórica da disciplina de Análise Numérica. A experiência prática reforçou que obter um número final na tela não é o suficiente; é imperativo garantir a robustez do código contra exceções lógicas (como matrizes singulares ou entradas incompatíveis), tratar as imprecisões residuais da linguagem de programação e dominar a capacidade de transformar problemas práticos em soluções computacionais autônomas, eficientes e confiáveis.